1 Step 1: Harmonization and Descriptive Analysis of PsychoPy Demographic Files

1.1 Objective

The objective of Step 1 is to create a clean, traceable, and fully documented demographic dataset from the PsychoPy outputs before any question-level or fNIRS-level analysis is performed. This step focuses only on the demographic and socioeconomic information collected before the task, together with the questionnaire metadata needed to interpret those responses correctly.

The present study investigates abstract versus concrete exam questions using fNIRS in an educational setting, and the project documentation states that participant information is pseudonymized and used in aggregated form. The demographics component therefore needs to be clean, consistent, and easy to join later with behavioral and fNIRS data. [**?**]

1.2 Current Raw Data Structure

Each participant appears only once in the dataset. Each participant has a dedicated folder named according to the pattern:

```
PID001/
PID002/
...
```

Inside each participant folder, at minimum, the following files are expected:

```
enem_blocks_PID01_20251106_124958.csv
socio_PID01_20251106_124958.json
```

The `enem_blocks` file contains event-level PsychoPy information, including the start of the questionnaire block and questionnaire item markers. The `socio` JSON file contains the actual demographic answers keyed by variable name.

In addition, the file `demographics_questions.json` defines the questionnaire schema, including question identifiers, prompts, variable types, required flags, and conditional display rules. This file should be treated as the primary questionnaire specification for Step 1. [**?**]

1.3 Principles for Step 1

Step 1 should follow six basic principles:

1. Keep the workflow simple and conservative.
2. Never overwrite or modify any raw file.
3. Preserve both raw values and harmonized values.
4. Treat `demographics_questions.json` as the reference schema for current sessions.

5. Flag ambiguous legacy values rather than forcing recodes.
6. Produce modular outputs: CSV tables, plots, and LaTeX snippets generated automatically.

1.4 Scope

This step includes:

- inventory of demographic-related raw files;
- extraction and validation of participant identifiers and timestamps;
- harmonization of demographic variables;
- grouped descriptive analysis of each demographic question;
- export of clean CSV files;
- generation of plots for each variable;
- generation of one LaTeX file per variable and one master LaTeX report.

This step does not include:

- analysis of question performance;
- reconstruction of missing fNIRS triggers;
- preprocessing or statistical analysis of the fNIRS signals.

1.5 Authoritative Variable List

The current questionnaire schema contains the following variables:

```
age
country_of_birth
nationality
years_in_luxembourg
gender
first_language
other_languages
affiliation
department_unit
programme_name
study_level
year_in_programme
employee_role
highest_degree
degree_field
country_secondary_ed
```

`primary_school_langs`
`parent_a_edu`
`parent_b_edu`
`first_gen_he`
`living_arrangement`

Some of these variables are conditional. For example, `programme_name`, `study_level`, and `year_in_programme` are displayed only if affiliation matches a student pattern, while `employee_role` is shown only for employee/staff/faculty/postdoc patterns. These conditions are explicitly defined in the questionnaire schema and must be respected during analysis. [**?**]

1.6 Question Groups

For organization and reporting, the variables should be analyzed in the following groups:

1. **Core identification and mobility variables**
   - `age`
   - `country_of_birth`
   - `nationality`
   - `years_in_luxembourg`
2. **Gender and language background**
   - `gender`
   - `first_language`
   - `other_languages`
   - `primary_school_langs`
3. **Current academic or professional status**
   - `affiliation`
   - `department_unit`
   - `programme_name`
   - `study_level`
   - `year_in_programme`
   - `employee_role`
4. **Educational background**
   - `highest_degree`
   - `degree_field`
   - `country_secondary_ed`
5. **Family educational background and living conditions**
   - `parent_a_edu`
   - `parent_b_edu`
   - `first_gen_he`
   - `living_arrangement`

1.7 Expected Inputs

The minimum raw inputs for Step 1 are:

- one participant folder per participant, e.g. `PID001/`;
- one PsychoPy event file per participant: `enem_blocks_PIDxx_YYYYMMDD_HHMMSS.csv`;
- one demographic response file per participant: `socio_PIDxx_YYYYMMDD_HHMMSS.json`;
- the questionnaire schema file: `demographics_questions.json`.

1.8 Expected Outputs

The following outputs should be generated in Step 1:

Structured tables

```
01_file_manifest.csv
02_participant_registry.csv
03_demographics_schema.csv
04_socio_raw_wide.csv
05_socio_raw_long.csv
06_socio_harmonization_dictionary.csv
07_socio_clean_wide.csv
08_question_summary.csv
09_question_level_counts.csv
10_question_missingness.csv
```

Plots

```
figures/step1/age_histogram.png
figures/step1/years_in_luxembourg_histogram.png
figures/step1/gender_barplot.png
figures/step1/affiliation_barplot.png
...
```

LaTeX outputs

```
reports/step1/questions/question_age.tex
reports/step1/questions/question_gender.tex
reports/step1/questions/question_affiliation.tex
...
reports/step1/step1_demographics_report.tex
```

1.9 Folder Structure for Step 1

A simple and organized structure is recommended:

```
project/
+-- data_raw/
|   +-- PID001/
|   +-- PID002/
|   +-- ...
+-- data_intermediate/
|   +-- step1/
+-- data_clean/
|   +-- step1/
+-- figures/
|   +-- step1/
+-- reports/
|   +-- step1/
|       +-- questions/
+-- scripts/
```

1.10 Detailed Workflow

1.10.1 Step 1.1: Build the raw file manifest

Create a manifest with one row per discovered file. For each file, record:

- participant folder name;
- participant ID extracted from folder;
- file name;
- file type (`enem_blocks`, `socio_json`, schema, other);
- timestamp extracted from the file name;
- full path;
- whether the expected pair exists.

The purpose of this file is to guarantee that every participant has the expected raw inputs and that the paired files refer to the same recording timestamp.

1.10.2 Step 1.2: Build the participant registry

Create a participant registry with exactly one row per participant. Since each participant appears only once, the participant registry should include:

- `participant_id` (e.g. `PID001`);

- folder name;
- PsychoPy file name;
- socio JSON file name;
- session timestamp;
- matching-status flag;
- notes.

Even though there is only one session per participant, it is still useful to keep the timestamp, because later steps will need to connect these files to the corresponding fNIRS recording.

1.10.3 Step 1.3: Parse the questionnaire schema

Read `demographics_questions.json` and convert it into a flat table called `03_demographics_schema.csv`. For each variable, store:

- question ID;
- question prompt;
- declared type;
- required flag;
- `show_if` dependency, if present;
- `show_if` regular expression, if present.

This table becomes the schema reference for all downstream validation and reporting. [**?**]

1.10.4 Step 1.4: Read the raw JSON answers

Read all `socio_PIDxx_timestamp.json` files into a single raw wide table. At this stage:

- do not recode any value;
- do not collapse categories;
- do not drop blank strings;
- keep exactly what was recorded.

This table should be saved as `04_socio_raw_wide.csv`.

1.10.5 Step 1.5: Convert the raw wide table to long format

Create a long-format version with one row per participant-variable pair. The minimum columns should be:

```
participant_id
timestamp
question_id
raw_value
source_json
```

This long table should be saved as `05_socio_raw_long.csv`. The long format will make grouped analysis, quality checks, and automated report generation much easier.

1.10.6 Step 1.6: Validate against the schema

For each participant and each schema variable, check:

- whether the variable exists in the JSON;
- whether the response type is compatible with the schema type;
- whether a missing value is allowed;
- whether the variable should have been shown according to the `show_if` rule.

This is very important for conditional variables. For example:

- if `affiliation` indicates a student, then `programme_name`, `study_level`, and `year_in_programme` should be evaluated as relevant fields;
- if `affiliation` indicates an employee, then `employee_role` becomes relevant;
- if a variable was not applicable, it should be coded as `NOT_APPLICABLE`, not as ordinary missing.

1.10.7 Step 1.7: Create the harmonization dictionary

Create `06_socio_harmonization_dictionary.csv`. This file must document every transformation rule. It should contain at least:

- variable name;
- raw value;
- harmonized value;
- transformation rule;
- comments;
- whether the rule is exact or approximate.

No value should be changed without an explicit rule in this dictionary.

1.10.8 Step 1.8: Apply basic text cleaning

For all text fields, apply only conservative cleaning:

- trim leading and trailing whitespace;
- collapse repeated spaces;
- standardize obvious capitalization;
- preserve accents;
- keep the original raw value in a separate column.

Examples:

- `" iran "` → `"Iran"`
- `"phd candidate"` → `"PhD candidate"`
- `"udm liser"` → `"UDM LISER"`

This step should remain conservative. No advanced NLP or semantic grouping should be introduced at this stage.

1.10.9 Step 1.9: Apply variable-specific harmonization

Each variable should then be harmonized according to its type.

Numeric variables The following variables should be parsed as numeric when possible:

- `age`
- `years_in_luxembourg`
- `year_in_programme`

Store:

- raw string value;
- parsed numeric value;
- validity flag.

Binary or small-category variables The following should be standardized to a small controlled vocabulary:

- `gender`
- `first_gen_he`

For example:

- `yes`, `Yes`, `Y` → `Yes`
- `no`, `No`, `N` → `No`
- unclear values → `Unknown`

Ordered education variables The following should be harmonized into ordered levels:

- `study_level`
- `highest_degree`
- `parent_a_edu`
- `parent_b_edu`

Open-text fields The following should remain essentially open-text, with only light cleaning:

- `country_of_birth`
- `nationality`
- `first_language`
- `other_languages`
- `department_unit`
- `programme_name`
- `degree_field`
- `country_secondary_ed`
- `primary_school_langs`
- `living_arrangement`

For these fields, the goal is not to aggressively merge categories at this stage. The goal is to remove trivial spelling noise while preserving real heterogeneity.

1.10.10 Step 1.10: Define missingness explicitly

Use explicit missingness labels:

- `MISSING`: expected but absent;
- `NOT_APPLICABLE`: hidden by questionnaire logic;
- `INVALID`: present but malformed;
- `UNKNOWN`: ambiguous and unresolved;
- `BLANK`: empty string recorded.

This distinction is important for descriptive analysis and later modeling.

1.10.11 Step 1.11: Build the clean wide table

Create `07_socio_clean_wide.csv`, with one row per participant and one harmonized column per variable. For each variable, it is recommended to keep:

- a harmonized analysis column;
- the raw value in a parallel column if needed;
- a flag column for missingness or uncertainty when useful.

1.10.12 Step 1.12: Produce grouped descriptive analysis for each question

Each demographic variable should receive its own descriptive analysis block.

For **numeric variables**, compute:

- number of valid responses;
- number of missing, invalid, and not-applicable cases;
- minimum;
- maximum;
- mean;
- standard deviation;
- median;
- quartiles.

For **categorical variables**, compute:

- number of valid responses;

- number of missing, invalid, and not-applicable cases;
- frequency of each harmonized category;
- percentage of each harmonized category;
- number of unique raw forms before harmonization.

For **free-text variables**, compute:

- number of valid responses;
- number of unique harmonized responses;
- top categories by frequency;
- optional grouped table if a simple manual recoding exists.

1.10.13 Step 1.13: Export per-question result tables

The per-question analysis should be saved into at least three files:

`08_question_summary.csv` One row per variable, containing:

- question ID;
- question prompt;
- variable group;
- variable type;
- number of participants;
- number of valid responses;
- number missing;
- number not applicable;
- number invalid;
- number of unique harmonized categories;
- notes.

`09_question_level_counts.csv` One row per variable-category pair, containing:

- question ID;
- harmonized category;
- count;
- percentage.

`10_question_missingness.csv` One row per variable and missingness type, containing:

- question ID;
- missingness label;
- count;
- percentage.

1.10.14 Step 1.14: Produce plots

Each variable should have its own plot.

Numeric variables Generate:

- histogram;
- boxplot if useful.

Categorical variables Generate:

- bar plot of harmonized category counts;
- optionally a second bar plot for missingness profile.

Open-text variables Generate:

- bar plot of the most frequent cleaned responses;
- truncate to top categories when necessary.

Plots should be saved in:

`figures/step1/`

1.10.15 Step 1.15: Generate one LaTeX file per variable

For each variable, generate a separate LaTeX file:

`reports/step1/questions/question_<id>.tex`

Each file should contain:

- subsection title with question prompt;
- variable metadata;

- short text summary;
- table of counts or summary statistics;
- inserted plot;
- notes on harmonization and missingness.

This modular structure allows the report to be rebuilt automatically without rewriting text by hand.

1.10.16 Step 1.16: Generate a master LaTeX report

Create a master file:

`reports/step1/step1_demographics_report.tex`

This file should:

- introduce the demographic module;
- describe the data sources;
- summarize the harmonization process;
- include grouped sections;
- load each variable file with `\input{...}`.

This makes the report easy to update whenever the data are refreshed.

1.11 Granular Reporting Template for One Variable

Each variable-specific LaTeX file should follow the same logic:

```
\subsection{Age}
\label{sec:q_age}

\paragraph{Question prompt.}
Age (in years).

\paragraph{Type.}
Numeric, required.

\paragraph{Availability.}
N participants, N valid, N missing, N invalid.

\paragraph{Summary statistics.}
Mean, SD, median, min, max, quartiles.
```

```
\paragraph{Quality notes.}
Any implausible values, formatting corrections, or exclusions.

\begin{figure}[ht]
    \centering
    \includegraphics[width=0.7\textwidth]{figures/step1/age_histogram.png}
    \caption{Distribution of age.}
\end{figure}
```

For a categorical variable, the structure should be analogous, replacing summary statistics with a frequency table.

1.12 Quality-Control Checks

Before considering Step 1 complete, the following checks should be run:

1. every participant folder has both expected files;
2. file timestamps match within participant;
3. schema variables are recognized;
4. no unexpected variables remain unmapped;
5. numeric fields parse correctly where expected;
6. conditional variables are correctly marked as applicable or not applicable;
7. frequencies are reviewed for suspicious spelling variants;
8. the final participant count matches the raw folder count.

1.13 Special Note on Legacy Sessions

The current questionnaire schema is text-based and uses named variables directly. However, earlier versions may have stored some answers as numeric option codes rather than text labels. If such legacy sessions exist, they must not be harmonized by guesswork. Instead, they should be decoded only from the original option mapping used at the time of collection. Until that mapping is available, such responses should be flagged as unresolved rather than forced into categories.

1.14 Minimal Completion Criteria for Step 1

Step 1 is complete when:

- all participant folders have been inventoried;
- all socio JSON files have been loaded;

- all variables have been checked against the questionnaire schema;
- a clean participant-level demographic table exists;
- grouped descriptive CSV outputs exist;
- one plot per variable exists;
- one LaTeX file per variable exists;
- one master demographic report exists;
- all harmonization decisions are documented.

1.15 Practical Interpretation

In simple terms, Step 1 should deliver one well-organized demographic analysis package: a clean participant table, a schema table, harmonization logs, grouped summaries, plots, and a modular LaTeX report that can be updated automatically whenever new participants are added.